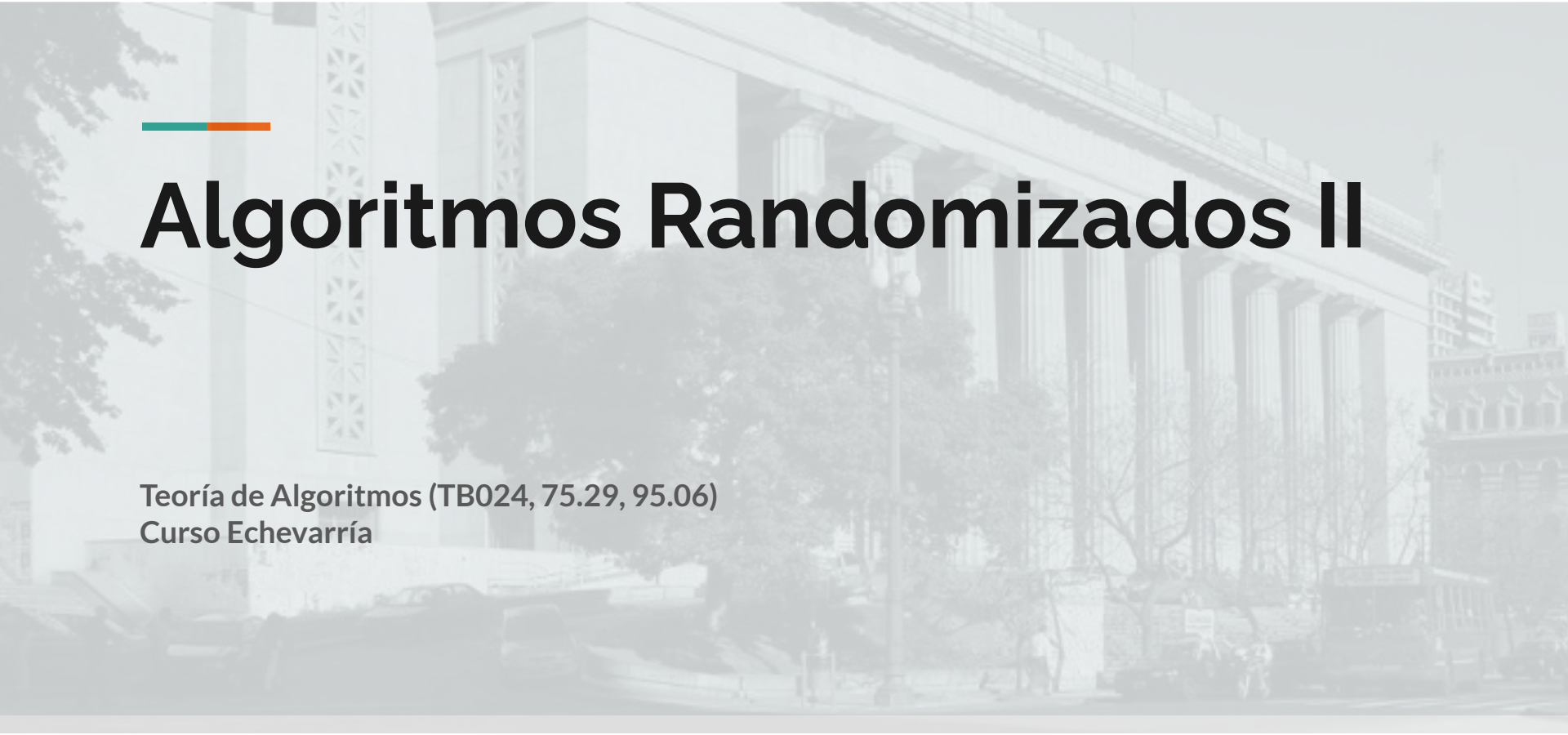


Algoritmos Randomizados II

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso de la Clase Anterior



Cottonbro Studio

Algoritmos randomizados

A veces, es necesario o conveniente introducir algún factor aleatorio en nuestros algoritmos.

Montecarlo vs. Las Vegas.

Aplicaciones:

- Resolución de colisiones
- Corte global mínimo
- MAX 3-SAT
- Cálculo de la mediana
- Quicksort

Aplicaciones: Skip Lists



Gaviotín lagunero

Skip Lists

Los arreglos ordenados nos permiten buscar en $O(log\ n)$

El mantenimiento de un arreglo ordenado requiere $O(n)$ para las operaciones de guardado y borrado

Podemos implementar una lista enlazada, logrando un tiempo $O(1)$ para agregar un elemento si sabemos su posición deseada

El problema es que encontrar elementos es $O(n)$

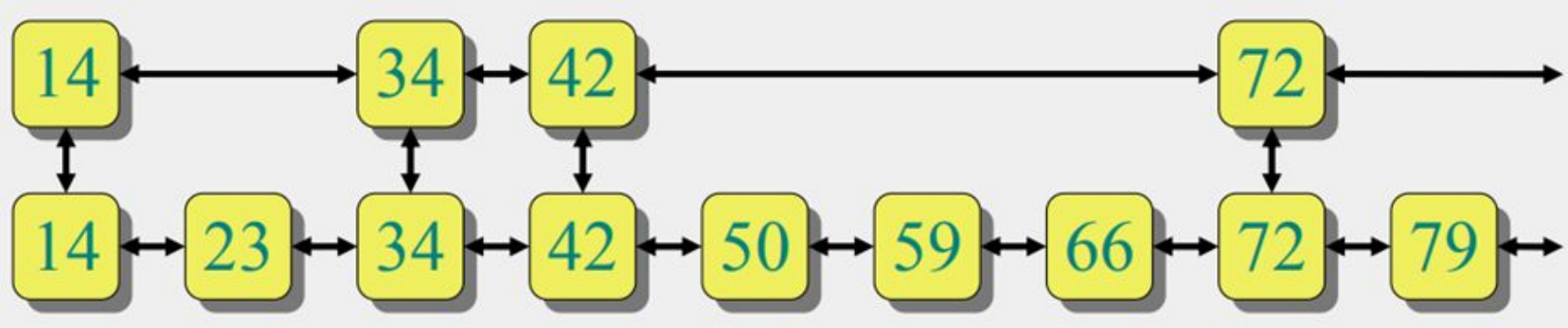


Skip Lists

Solución: ¿más listas?

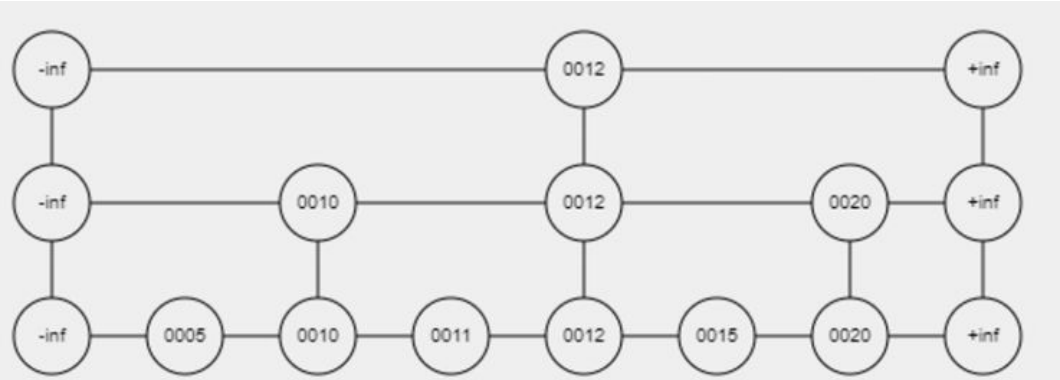
Las Skip Lists son estructuras que incorporan una randomización para permitir de forma eficiente, con **alta probabilidad**, la búsqueda, guardado y borrado en una lista ordenada

https://cmps-people.ok.ubc.ca/ylucet/DS/SkipList.html



Skip Lists

- La idea es realizar una búsqueda a partir de la capa superior de la lista, imitando el movimiento en un árbol binario.
- Al insertar siempre se insertará en la base, y se sube de capa con probabilidad p
- Las capas superiores tienden a tener menos elementos. Al buscar cualquier número mayor a 12, se hace “skip” de todos los anteriores.
- Las capas superiores funcionan como ruta express para llegar al elemento.



Skip Lists

Orden temporal promedio, con **alta probabilidad**: $\frac{1}{p} \log_{1/p} n$

- Búsqueda: $O(log\ n)$
- Inserción: $O(log\ n)$
- Borrado: $O(log\ n)$

Orden temporal de los peores casos: $O(n)$

Orden espacial en el peor caso: $O(n\ log\ n)$

Skip Lists

Orden temporal promedio: $\frac{1}{p} \log_{1/p} n$

En promedio, cada elemento está presente en $1/(1-p)$ listas.

La cantidad esperada de listas es: $\log_{1/p} n$

Es posible modificar el valor de p para regular el uso de memoria y la velocidad esperada de las operaciones.

A mayor p, mayor uso de memoria y velocidad.

Aplicaciones: Treaps



Picabuey

Árboles binarios de búsqueda

- Permiten mantener dinámicamente una estructura de tipo diccionario, permitiendo además presentar los datos según una relación de orden
- Generalmente sus operaciones son $log(n)$... Siempre y cuando estén balanceados
- Es por eso que hay varias metodologías de balanceo de árboles
  - -AVL
  - -Red-Black
- “La altura esperada de un árbol creado en orden aleatorio es $O(log\ n)$”

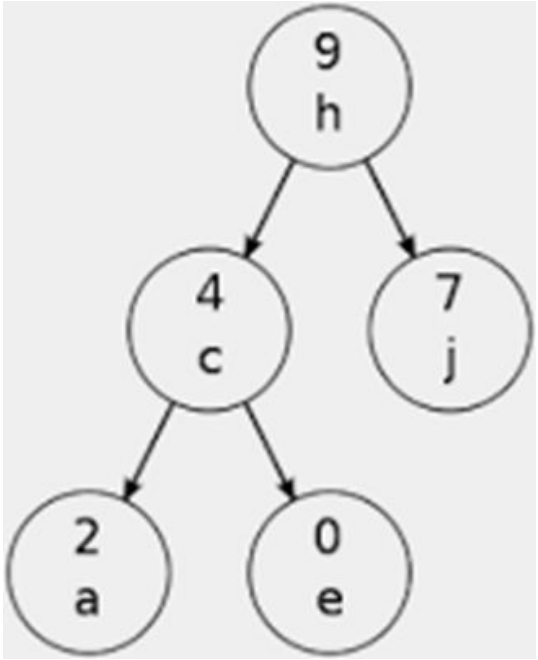
Árboles binarios de búsqueda

- Si la altura esperada de un árbol creado en orden aleatorio es $O(log\ n)$, mantengamos un árbol como si fuese creado en orden aleatorio
- Treap = Tree + Heap

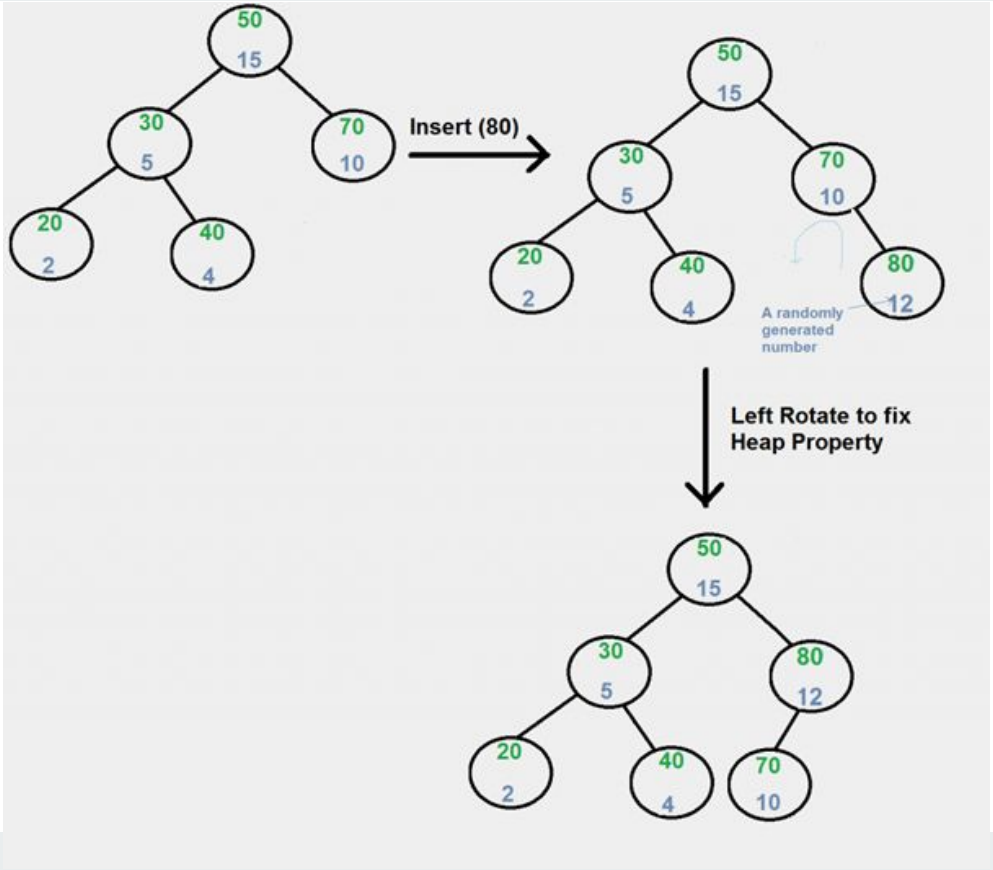


Treaps

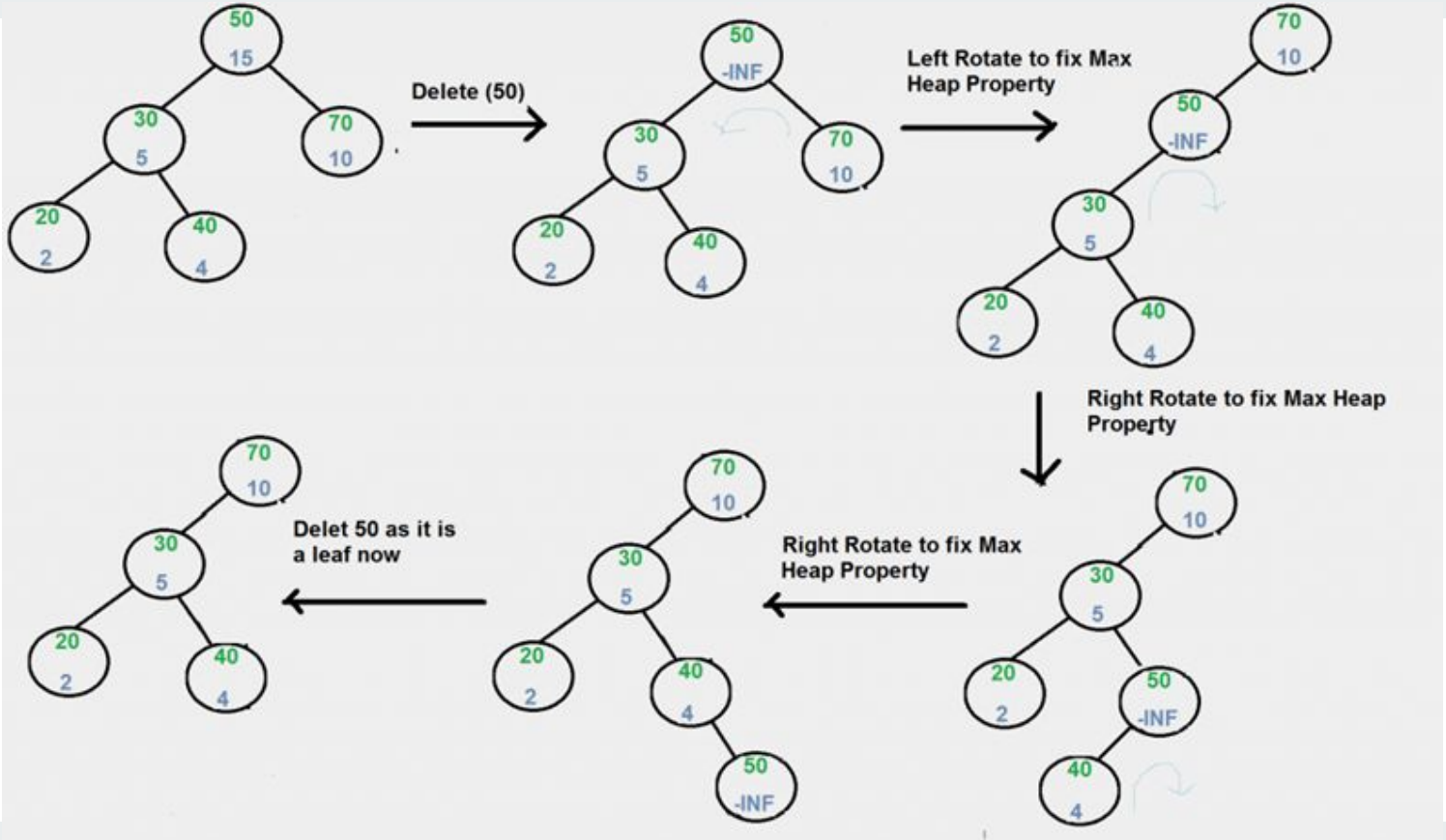
- Cada nodo tiene la clave correspondiente al árbol
- Además tiene una prioridad que es calculada de manera aleatoria
- La estructura debe mantener ambos datos, las claves en orden, y las prioridades en propiedad de parva
- El orden aleatorio de las prioridades mantendrá balanceado el árbol con alta probabilidad
- Por lo tanto, son esperables operaciones $O(\log n)$
- Es como si los nodos se hubieren insertado primero a mayor prioridad (siendo la prioridad al azar)



Treaps


50
15
30
5
70
10
20
2
40
4
Insert (80)
50
15
30
5
70
10
20
2
40
4
80
12
A randomly generated number
Left Rotate to fix Heap Property
50
15
30
5
80
12
20
2
40
4
70
10

Treaps


50
15
30
5
70
10
20
2
40
4
Delete (50)
50
-INF
30
5
70
10
20
2
40
4
Left Rotate to fix Max Heap Property
70
10
50
-INF
30
5
20
2
40
4
Right Rotate to fix Max Heap Property
70
10
30
5
20
2
50
-INF
40
4
Right Rotate to fix Max Heap Property
70
10
30
5
20
2
40
4
50
-INF
Delet 50 as it is a leaf now
70
10
30
5
20
2
40
4

Aplicaciones: Árboles Binarios de Búsqueda Aleatorios



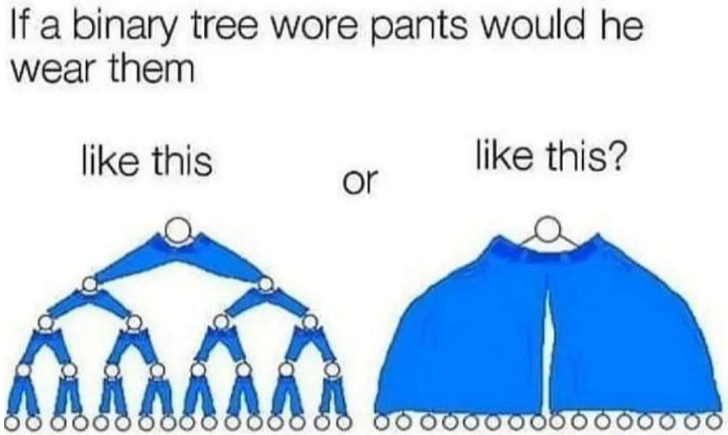
Árboles binarios de búsqueda aleatoria

- Nuevamente aprovecha que la altura esperada de un árbol creado en orden aleatorio es $O(\log n)$.
- Lo hace de una forma bastante más literal.
- Cuando hay que hacer una inserción, por cada nodo que se visita se lanza una moneda con cierta probabilidad, en caso de éxito corresponde insertar ese elemento en esa posición del árbol.
- Para mantener la uniformidad de la probabilidad p, cada nodo mantiene la cantidad k de nodos descendientes (contándose a sí mismo).
- La probabilidad p en cada paso de la inserción es $1/(k+1)$.



Árboles binarios de búsqueda aleatoria

- La probabilidad p en cada paso de la inserción es $1/(k+1)$
  Ejemplo: al comenzar a insertar por la raíz, la probabilidad de que el nuevo elemento esté en la raíz es igual a la probabilidad de que, en un orden aleatorio para estos $n+1$ elementos, el nuevo elemento haya sido el primero.
- Por lo tanto, $p = 1/(n+1)$.



Árboles binarios de búsqueda aleatoria

- Este análisis se transporta análogamente a cada subárbol, y por ello cada nodo necesita almacenar el valor de la cantidad de descendientes.
- Nuevamente, es equivalente a crear un árbol en un cierto orden aleatorio, en este caso determinado por los resultados de las evaluaciones de las probabilidades p, que equivalen a cierta permutación de todas las posibles para estos elementos.

Generación de Números Pseudoaleatorios



Martín pescador chico

Generación de números pseudoaleatorios

Se busca generar una secuencia de números que no pueda predecirse mejor que por mero azar

Métodos de generación:

- True random: en base a mediciones de Hardware de procesos impredecibles
- Pseudo random: algoritmos que emulan el azar mediante procesos determinísticos dada una semilla inicial

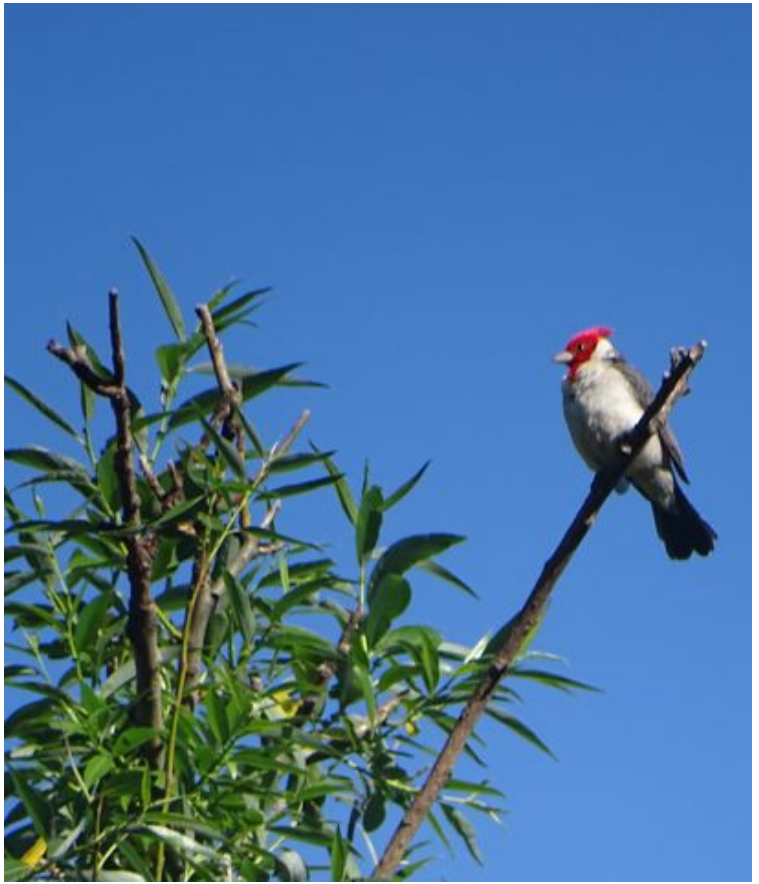
Generación de números pseudoaleatorios

- Generación determinística de números aleatorios.
- Es un algoritmo que genera una secuencia de números con propiedades aproximadamente similares a las que generaría un verdadero random.
- Es determinístico para una semilla (seed).
- Tiene la capacidad de generar números a una velocidad mucho mayor.
  - Cuadrados medios
  - Se debe demostrar distribución equiprobable

```
#include <iostream>
using namespace std;
#include <ctime>

void main()
{
   int n;
   long i;
   srand(time(NULL));
   n=1;
   do
   {
      n = rand()%37;
      cout << n << "   ";
      for (i=0; i<100000000; i++);
   }
   while (n!=0);
   cin>>n;
}
```

Conteo Aleatorio



Cardenal copete rojo

Conteo aleatorio

Si yo quiero contar la cantidad de apariciones de cada uno de los posibles elementos que haya, en una secuencia de elementos (ej palabras), ¿cuánto me consumirá en memoria?

- Counting Filters: primer algoritmo utilizado por twitter para determinar los TT.
- Depende fuertemente de las funciones de hashing.

Supongamos que ahora sólo quiero contar la cantidad de elementos/palabras diferentes que hay… y además tenemos una memoria limitada…

¿Y si contamos completamente de forma aleatoria?

Conteo aleatorio

Supongamos que podemos tener en memoria $m = 100$ elementos (palabras).

Los primeros 100 que nos topemos, lo guardamos, perfecto. Una vez que llegamos a 100, por cada elemento tiramos una moneda, si sale cara lo mantenemos, si sale cruz lo olvidamos. ¿Cuántos elementos esperamos tener cuando termine este proceso?

Ahora proseguimos, pero cada vez que me topo con una palabra ya vista tiramos una moneda para ver si la mantenemos o borramos.

Conteo aleatorio

Si en cualquier momento llegamos a 100 palabras, volvemos a aplicar el proceso anterior.

Si aplicamos ese segundo procesamiento, entonces para mantener una palabra en memoria no lanzaremos una moneda, sino 2 (y ambas deben salir cara para mantener).

Si se aplica el procesamiento de 100 otra vez, serán 3 monedas, y así.

Con este método, nos aseguramos que cada palabra tenga probabilidad $(1/2)^K$ de estar en nuestro listado, con K = cantidad de veces que se hizo el procesamiento al llegar a $m = 100$.

Conteo aleatorio

Supongamos que nos tomó 6 rondas completar de procesar Hamlet. Terminamos con 61 palabras en nuestro listado → la cantidad de palabras estimadas es $61 * 2^6 = 3904$. Hamlet tiene 3967, menos de 2% de error.

Si cambiamos $m = 100$ por $m = 1000$, el resultado llega a ser aprox de 3964 (menos de 0.01% de error).

Si m = suficientemente grande para que entren todas las palabras, la precisión será del 100%.

Hashing



Diccionarios

Organizar la información en diccionarios, nos debe permitir:

- Guardar un elemento u
- Buscar un elemento u
- Borrar un elemento u
- Determinar si un elemento u pertenece al diccionario

Queremos hacer esto rápidamente.
Si la cantidad de elementos posibles del universo es reducida (n), podemos implementarlo en un array de n elementos.
Si la cantidad es muy grande (U), esto ya no es posible.

Diccionarios y Hashing

Los Diccionarios resuelven este problema mediante una función que transforma el elemento u a un valor numérico, posición de un arreglo, la **Función de Hashing**

$h: U \rightarrow \{0, 1, ..., n\text{-}1\}$ para indexar la tabla de Hash

Sería genial que para $u \neq v$, $h(u) \neq h(v)$. Si no → colisión

Nuestras funciones de Hashing deben distribuir los elementos del Universo U de la mejor manera posible en las n posiciones de nuestra tabla de Hash

Problema: para cualquier $|U| > n^2$, existen n elementos que coinciden en el valor de la función h, creando colisión para todos ellos, y se degrada la complejidad a $O(n)$

Funciones de Hash

- Podríamos usar los primeros bits de cada elemento: dichos bits, según el dominio del problema podrían repetirse con frecuencia.
  - DNI
- Tomar siempre módulo n: ídem, especialmente para potencias de 2
- En general, cualquier selección fija puede estar consultando algún tipo de propiedad sobre los elementos utilizados, y generar una mala distribución.
- Tomar siempre módulo p para algún número primo: no necesariamente funciona en toda aplicación, y no funciona bien para primos cercanos a potencias de 2.
- Hay muchas funciones que han probado funcionar a nivel empírico.
- Vamos a ver cómo crear funciones que teóricamente funcionan bien.

Funciones de Hash

Ejemplo para definir $h(u)$:

- Para insertar un elemento u, tomamos $h(u)$ un valor aleatorio entre 0 y $n-1$.
- La probabilidad de colisión es baja, y se usa todo el espacio del array.
- Ahora, ustedes definan la primitiva `determinar_si_existe(u)`



Clases de Funciones de Hash

Vamos a elegir, al azar, una función de Hash dentro de una Clase de Funciones $\mathcal{H}$

Propiedades deseadas:

1. Para cualquier par de elementos u,v en el Universo, la probabilidad de colisión es como mucho $1/n$ → La Clase es Universal
2. La función h puede ser representada de forma compacta, y podemos calcular el valor $h(u)$ de manera eficiente

Clases de Funciones de Hash

Si la Clase es Universal, las funciones h dentro de la clase cumplen la propiedad:

Para cualquier Set S en U de tamaño hasta n, sería esperable que la cantidad de elementos en S que colisionan un elemento u sea a lo sumo 1

$$E\left[X_s\right] = \Pr\left[X_s = 1\right] \leq 1/n$$

$$E\left[X\right] = \sum_{s \in S} E\left[X_s\right] \leq |S| \cdot \frac{1}{n} \leq 1$$

Clase Universal

Buscamos un número primo p similar al valor de n como tamaño de tabla de Hash

Cada función dentro de la clase se identifica como $a = (a_1, a_2, \ldots, a_r)$ con $0 \leq a_i \leq p$

Para Universo de N elementos, $r \approx log\ N / log\ n$, tomando un conjunto de bits en c/u

Vamos a definir el valor de F, Hash para algún vector x, con $0 \leq x_i \leq p$

La representación de cada F de Hash cumple propiedad 2, es compacta

$$h_a(x) = \left(\sum_{i=1}^{r} a_i x_i \right) \bmod p$$

Clase Universal

Podemos probar que definir nuestra clase de funciones de esta manera y ver si la hace una Clase Universal

Tendríamos que ver que para cualquier par de elementos $x = (x_1, x_2,..., x_r)$, $y = (y_1, y_2,..., y_r)$ tenga como mucho una probabilidad $1/p$ de tener una colisión, que $h(x)=h(y)$

Siendo que x e y no son el mismo elemento, existe al menos un j tal que $x_j \neq y_j$

La selección de nuestra función h requiere seleccionar cada uno de los valores de a: $(a_1, a_2,..., a_r)$. Nos centramos en la coordenada a_j donde sabemos difieren x e y

Clase Universal

Nos centramos en la coordenada a_j donde sabemos difieren x e y

Sin importar el resto de los valores de a, la probabilidad de colisión sería $1/p$

Veamos el valor de a_j. La colisión se da cuando $h_a(x) = h_a(y)$

$$h_a(x) = \left(\sum_{i=1}^{r} a_i x_i\right) \bmod p \quad h_a(y) = \left(\sum_{i=1}^{r} a_i y_i\right) \bmod p$$

$$a_j(y_j - x_j) = \sum_{i \neq j} a_i(x_i - y_i) \bmod p$$

Clase Universal

$$a_j(y_j - x_j) = \sum_{i \neq j} a_i(x_i - y_i) \bmod p$$

Del lado derecho la ecuación da algún número $m \ mod \ p$

¿Cuántos valores de a_j satisfacen esta igualdad?

Clase Universal

$$a_j(y_j - x_j) = \sum_{i \neq j} a_i(x_i - y_i) \bmod p$$

Del lado derecho la ecuación da algún número $m \ mod \ p$

¿Cuántos valores de a_j satisfacen esta igualdad?

Con $z=y_j-x_j$, la igualdad es $a_j z = m \ mod \ p$

Siendo que $0 \leq a_j \leq p$, y que p es un número primo, hay solamente un valor que satisface la igualdad, valor seleccionado con una probabilidad de $1/p$

Ejercicios para la clase que viene:

2-Partition Problem

- Se tiene un conjunto S de números
- Encontrar 2 particiones P_1 y P_2 de S tales que la suma de los elementos de cada una sea la mínima posible
- Algoritmo Greedy propuesto:
  - Inicializar P_1 y P_2 como dos sets vacíos
  - Mientras queden elementos en S:
    - Tomar el próximo elemento $s \in S$
    - Agregarlo al set P_1 o P_2 que tenga mínima suma hasta el momento
  - Devolver P_1 y P_2
- Mostrar que el algoritmo Greedy propuesto es una 2-aproximación

Ejercicios para la clase que viene:

Proponer un algoritmo randomizado de tipo “Las Vegas” para resolver el problema de N-Reinas.

- Puede encontrar una solución, o fallar
- Se puede repetir el ensayo muchas veces hasta que no falle y devuelva una solución
- ¿Cuál es la cantidad esperada de veces que hay que repetir el ensayo para encontrar una solución?



¡Muchas gracias!